IT305 Lab: Socket Programming Project

Fault Tolerant Topic-based File Distribution Service

Design Document

Submission 1

Project Group 016_024

#	Name	Roll number
1	Jhanvi Barot	202401076
2	Priyanshi Kotadiya	202401097
3	Nidhi Chauhan	202401128
4	Het Lathiya	202401104
5	Rushiraj Navar	202401127
6	Om Pandya	202401136

Contents

1 Introduction **3**

2 Overview **3**

2.1 Language and platform . . . 3

2.2 Setup for experiments . . . 3

2.3 Command line . . . 4

2.4 Fixed values . . . 5

3 Server Data Structures **5**

3.1 Topic table . . . 5

3.2 Session table (Case 2 and Case 2 Enhanced) . . . 6

4 Message Formats **6**

4.1 General rules . . . 6

4.2 Part I . . . 7

4.3 Part II, Case 1 . . . 8

4.4 Part II, Case 2 . . . 8

4.5 Part II, Case 2 Enhanced . . . 8

5 Server and Client Structure **9**

5.1 Single-threaded and multi-threaded server . . . 9

5.2 Shared data and locking . . . 10

5.3 Socket settings . . . 10

5.4 Client . . . 10

6 Fault Injection **10**

7 Session Management (Case 2) **11**

7.1 What each side stores . . . 11

7.2 Positions . . . 12

7.3 Message sequence . . . 12

7.4 Checking a RESUME request . . . 13

7.5 A RESUME while the old connection is still active . . . 13

7.6 Client procedure . . . 14

7.7 Why no saved data is sent again . . . 14

8 Performance Enhancements (Case 2 Enhanced) **15**

8.1 Socket mode . . . 15

8.2 Parallel TCP connections . . . 15

8.3 How the improvement will be shown 16

9 Performance Measurement 16

9.1 What is measured 16
9.2 Metrics 16
9.3 Experiments 17
9.4 Output files 17
9.5 What we expect 17

10 Additional Features 18

11 Code Structure 18

11.1 Folders and files 18
11.2 Main functions 18
11.3 How the parts fit together 19
11.4 Build 19

12 Design Thought Process 19

12.1 Part I 19
12.2 Case 1 19
12.3 Case 2 19
12.4 Case 2 Enhanced 20

13 Assumptions and Limitations 20

1 Introduction

This document describes how we plan to build the topic-based file distribution service for the IT305 project. The project has four parts that build on each other:

(a) **Part I**: a TCP server that sends all files of a requested topic, written in a single-threaded and a multi-threaded version;
(b) **Part II, Case 1**: the multi-threaded server with fault injection; after a failure the transfer starts again from the beginning;
(c) **Part II, Case 2**: the same, but with session management, so the client resumes from its last checkpoint;
(d) **Part II, Case 2 Enhanced**: Case 2 with parallel TCP connections to improve performance.

Nothing has been implemented or measured yet. Everything below is the plan we will follow for the final submission, and any change we make during implementation will be reported in the final report.

The assignment lists the topics this document must cover. Table 1 shows where each one is discussed.

Table 1: Where each required topic is covered.

Required topic	Section
Server data structures that map topics to directories	3
Message formats for Part I, Case 1, Case 2 and Case 2 Enhanced	4
Fault injection mechanism	6
Session management protocol	7
Planned performance enhancements	8
Additional features and why we add them	10
Code structure and how the parts fit together	11
Design thought process and alternatives	12
Performance measurement plan	9

2 Overview

2.1 Language and platform

All programs will be written in C (the C11 standard with GNU extensions, compiled with `gcc -std=gnu11`) using POSIX TCP sockets (IPv4) and the pthread library, and will run on the Linux machines in the lab. The GNU mode makes POSIX functions such as `realpath()` and `erand48()` available without extra feature-test macros. Time is measured with `clock_gettime(CLOCK_MONOTONIC)`. We do not use any external library. Each of the four folders (`Part1`, `Part2_Case1`, `Part2_Case2`, `Part2_Case2_Enhanced`) has its own `Makefile`, and running `make` in a folder builds exactly two executables in that folder: `server` and `client`.

2.2 Setup for experiments

We use three computers connected to the lab LAN (Figure 1). Node A runs the server. Nodes B and C each run several client processes at the same time. The server reads the dataset from the folder given on its command line: the extracted Animals-10 folder, which has one sub-folder per animal.

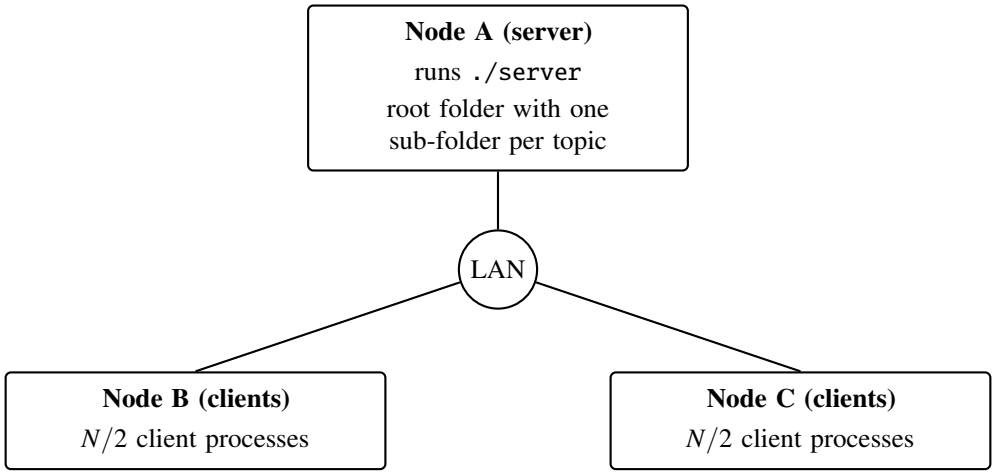


Figure 1: Machines used for the experiments.

2.3 Command line

The programs follow the command-line interface given in the assignment exactly. Nothing (paths, ports, addresses) is hard coded. A few optional arguments may be added after the required ones. They are written as `key=value`, and all of them have default values, so the required form always works on its own.

Table 2: Program invocations.

Program	Invocation
Server, Part I	`./server <port> <topics_root_dir> [mode=single|multi] [csv=<file>]`
Server, Part II (all cases)	`./server <port> <topics_root_dir> <failure probability> [seed=<n>] [csv=<file>]`
Client, all parts	`./client <server_ip> <server_port> <topic> <output_dir> [csv=<file>]`
Client, Case 2 Enhanced	same as above, plus optional `[conns=<k>]`

Table 3: Optional arguments.

Key	Used by	Default	Meaning
`mode`	server, Part I	`multi`	`single`: clients are served one after another; `multi`: one thread per client.
`seed`	server, Part II	current time	Seed for the random numbers used in fault injection. The value used is printed when the server starts.
`csv`	server and client	not set	If given, one line of measurements is appended to this CSV file per connection (server) or per transfer (client).
`conns`	client, Case 2 Enhanced	4	Number of parallel TCP connections to use (1 to 8).

The server checks that the port is between 1 and 65535, that the root folder can be opened, and that the failure probability is a number between 0 and 1. The client resolves `<server_ip>` with `getaddrinfo()`, so both an IPv4 address and a host name work. If any argument is wrong, the program prints a usage line and exits with status 1.

2.4 Fixed values

Some values are fixed in the code as constants rather than passed on the command line.

Table 4: Constants used in all parts.

Constant	Value	Purpose
CHUNK_SIZE	64 KiB	File data read and sent at a time (sent with a loop around send(), since one call may send fewer bytes).
MAX_LINE	1024 bytes	Longest allowed protocol line.
IO_TIMEOUT	30 s	Socket send and receive timeout.
RETRY_LIMIT	100	Connection attempts allowed per transfer (per stream in Enhanced).
RETRY_DELAY	100 ms	Wait between connection attempts.
MAX_STREAMS	8	Largest number of parallel connections (Enhanced).

3 Server Data Structures

3.1 Topic table

When the server starts, it reads <topics_root_dir> once and builds a table of topics. Every sub-folder becomes a topic. The key is the sub-folder name exactly as it appears on disk, and the value is the absolute path of that folder, obtained with realpath(). For the dataset this gives ten entries: cane, cavallo, elefante, farfalla, gallina, gatto, mucca, pecora, ragno and scoiattolo, for example cane → <absolute root>/cane. This is the same mapping as in the assignment's table, where the value is /home/manish/lab-6/raw-img/cane; our value depends on where the dataset is placed. Regular files in the root folder are not topics and are ignored. This covers, for example, the translate.py file included in the dataset ZIP if it is placed there. The English names listed in that file (dog, horse, ...) are not topic names.

For every topic we also store the list of files in the folder (name and size), sorted by name. We call this list the *manifest*.

```
struct file_entry {
    char     name[256];          /* file name inside the topic folder */
    uint64_t size;               /* size in bytes                     */
};

struct topic {
    char     key[256];           /* topic name = sub-folder name      */
    char     path[PATH_MAX];     /* absolute path of the folder       */
    uint32_t nfiles;
    uint64_t total_bytes;        /* sum of all file sizes             */
    struct file_entry *files;    /* sorted by name                    */
};

struct topic_table {
    uint32_t      ntopics;
    struct topic *topics;        /* sorted by key                     */
};
```

How the table is built:

1. Open the root folder with opendir() and go through it with readdir(). Every entry that stat() reports as a directory (except . and ..) becomes a topic. A folder whose name contains a space or a

newline is skipped with a warning, because such a name cannot be sent in a request line.

2. In each topic folder, every entry that `stat()` reports as a regular file (including hidden ones) is added to the manifest with its size, so the client receives every file of the folder. Sub-folders inside a topic are ignored. A file name containing a newline is skipped with a warning.
3. Sort each manifest by file name and sort the topics by key, both with `strcmp()`.

To serve **GET** <topic>, the server finds the topic with a binary search on the key (`bsearch()`), comparing names exactly. If there is no exact match, it compares the name once more ignoring upper and lower case (`strcasecmp()`), so a request for `Cane`, as written in the assignment's table, still finds `cane`. This is used only when exactly one topic matches. If the topic is still not found, the server replies with **ERR** 404.

We chose this structure for three reasons. First, the table does not change after start-up, so all threads can read it at the same time without any lock. Second, a sorted manifest gives every file a fixed index (0, 1, 2, ...), which the resume protocol in Case 2 relies on. Third, knowing all file sizes in advance lets the server tell the client the total size before sending anything. A new topic can be added by creating a new sub-folder and restarting the server; no code changes are needed.

3.2 Session table (Case 2 and Case 2 Enhanced)

For session management the server also keeps a table of sessions. This is the only data that several threads modify, so it is protected by one mutex.

```
struct session {
    uint64_t      sid;                  /* session id                 */
    struct topic *topic;
    uint32_t      k;                    /* streams (always 1 in Case 2) */
    uint32_t      sent_idx[MAX_STREAMS]; /* high-water mark per stream: */
    uint64_t      sent_off[MAX_STREAMS]; /*   furthest position sent   */
    uint32_t      gen[MAX_STREAMS];     /* owner number of each stream */
    int           done[MAX_STREAMS];    /* BYE received for stream    */
    struct session *next;
};

struct session *session_list;           /* linked list of sessions    */
pthread_mutex_t session_lock;           /* protects the whole list    */
```

A linked list is enough here because the number of sessions is small (one per client transfer) and the list is searched only once per connection, not once per chunk. Sessions stay in memory until the server exits. Each one takes about two hundred bytes, so even thousands of transfers use very little memory, and we avoid the extra work of freeing sessions that another thread might still be using.

4 Message Formats

4.1 General rules

- Each client connection carries one request and the reply to it.
- Control messages are lines of ASCII text. Fields are separated by one space and the line ends with a newline character (\n). A line may be at most `MAX_LINE` bytes long.
- File data is sent as raw bytes directly after a **FILE** line. The **FILE** line gives the size, so the receiver knows exactly how many bytes to read before the next line starts. No escaping is needed.
- The file name is always the last field of a **FILE** line and runs to the end of the line, so names that contain spaces still work.

- Both sides read lines through a small buffered reader, because one `recv()` may return part of a line, or a line followed by file data.

The fields used in the messages are:

Table 5: Message fields.

Field	Meaning
<topic>	Topic name (sub-folder name).
<nfiles>	Number of files in the topic.
<total>	Total size of all files of the topic in bytes.
<idx>	Index of a file in the sorted manifest, starting from 0.
<offset>	Byte position in the file from which the following data starts.
<size>	Size of the whole file in bytes.
<name>	File name.
<sid>	Session id, written as 16 hexadecimal digits (Case 2 and Enhanced).
<k>	Number of parallel connections (Enhanced).
<stream>	Number of a connection within a session, from 0 to $k-1$ (Enhanced).

Errors are reported with **ERR** <code> <text>, after which the server closes the connection:

Table 6: Error codes.

Code	When it is sent	Parts
400	The request line is not valid.	all
404	The topic does not exist.	all
409	A **RESUME** checkpoint is not valid (Section 7.4).	Case 2, Enhanced
410	The session id is unknown or the session is already finished.	Case 2, Enhanced
500	A file of the topic cannot be opened; sent in place of that file's **FILE** line.	all

When the client receives **ERR**, it prints the message and exits with status 1. It does not retry, because retrying would give the same answer.

4.2 Part I

Table 7: Part I messages (C = client, S = server).

From	Message	Meaning
C to S	**GET** <topic>	Ask for all files of the topic.
S to C	**OK** <nfiles> <total>	Topic found. Files follow.
S to C	**FILE** <idx> 0 <size> <name>, then <size> bytes	One file. Files are sent in manifest order, $idx = 0, 1, \ldots, nfiles - 1$.
S to C	**END**	All files have been sent.
S to C	**ERR** <code> <text>	Instead of **OK**, or instead of a **FILE** line (500).

In Part I the offset in a **FILE** line is always 0. We still include the index and the offset so that the same **FILE** line and the same receiving code can be used in every part. In Part I the client only checks that the index goes up by one each time and that the offset is 0.

After receiving **END** the client closes the connection. The server stops its timer as soon as the **END** line has been sent. It then waits for end-of-file from the client and closes its side, but this wait is not part of the measured time, so the time depends only on the server's sending. At that moment the last few bytes may still be in the kernel's send buffer, which is small compared with the size of a topic.

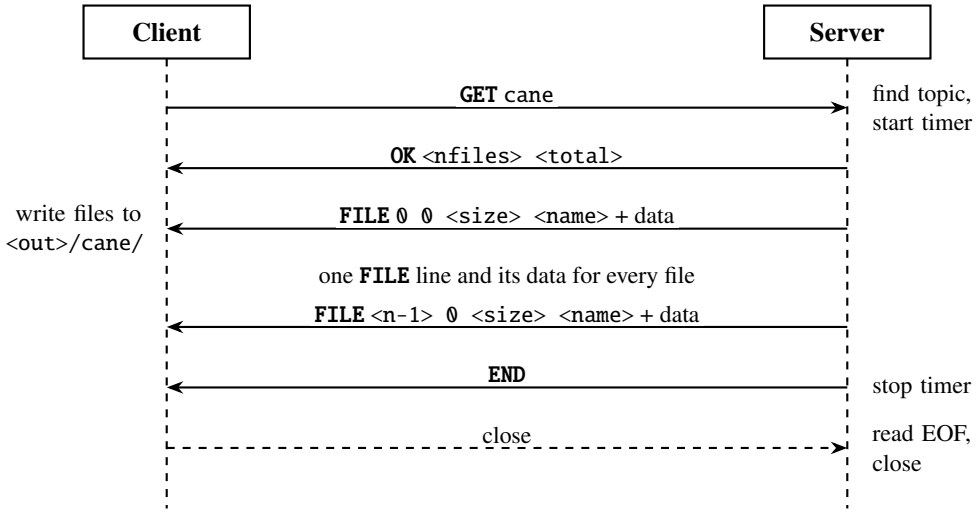


Figure 2: Message exchange in Part I.

4.3 Part II, Case 1

Case 1 uses exactly the same messages as Part I. The difference is only in behaviour. The server may break the connection while sending data (Section 6). When that happens, the client closes its socket, waits `RETRY_DELAY`, connects again and sends the same **GET** <topic>. The server then sends the whole topic again from the first file. No progress information is kept by either side.

4.4 Part II, Case 2

Table 8: Case 2 messages.

From	Message	Meaning
C to S	**GET** <topic>	Start a new session.
S to C	**SESSION** <sid> <nfiles> <total>	Session created. Files follow from file 0.
C to S	**RESUME** <sid> <topic> <idx> <offset>	Continue session <sid> from the client's checkpoint.
S to C	**RESUMED** <sid> <idx> <offset>	Checkpoint accepted. Data follows from that point.
S to C	**FILE** <idx> <offset> <size> <name>, then <size> − <offset> bytes	Part of a file (after a resume) or a whole file.
S to C	**END**	The rest of the topic has been sent.
C to S	**BYE**	All files are saved. The session is finished.
S to C	**ERR** <code> <text>	Instead of **SESSION** or **RESUMED**, or instead of a **FILE** line (500).

Only the first **FILE** line after **RESUMED** can have an offset larger than 0. All later ones have offset 0.

4.5 Part II, Case 2 Enhanced

In the enhanced version a transfer uses k connections, which we call streams. File number i is sent on stream i mod k. The messages are those of Case 2 with a stream count and a stream number added.

Table 9: Case 2 Enhanced messages.

From	Message	Meaning
C to S	**GET** <topic> <k>	New session with k streams. This connection is stream 0. Without <k>, $k = 1$.
S to C	**SESSION** <sid> <nfiles> <total> <k>	Session created. The server may lower k to MAX_STREAMS. Files of stream 0 follow.
C to S	**RESUME** <sid> <topic> <stream> <idx> <offset>	Connect a stream to the session. Used both to open streams $1, \ldots, k-1$ (checkpoint <stream> and 0) and to reconnect a stream after a failure.
S to C	**RESUMED** <sid> <stream> <idx> <offset>	Accepted. Files of this stream follow.
S to C	**FILE**, **END**	As in Case 2, but only the files of this stream ($idx \bmod k = stream$).
C to S	**BYE**	All files of this stream are saved.

5 Server and Client Structure

5.1 Single-threaded and multi-threaded server

Both versions of the Part I server call the same function, `handle_client()`, which reads the request, sends the reply and closes the connection. They only differ in how that function is called:

- **Single-threaded** (`mode=single`): the main thread calls `accept()`, then `handle_client()`, then goes back to `accept()`. Clients that connect in the meantime wait in the listen queue and are served in order.
- **Multi-threaded** (`mode=multi`, and every Part II case): the main thread only calls `accept()`. For every new connection it creates a detached thread with `pthread_create()`, and that thread runs `handle_client()`. This is the one-thread-per-client design that Part II asks for.

Because both versions use the same handler, the comparison in Part I measures only the effect of threading.

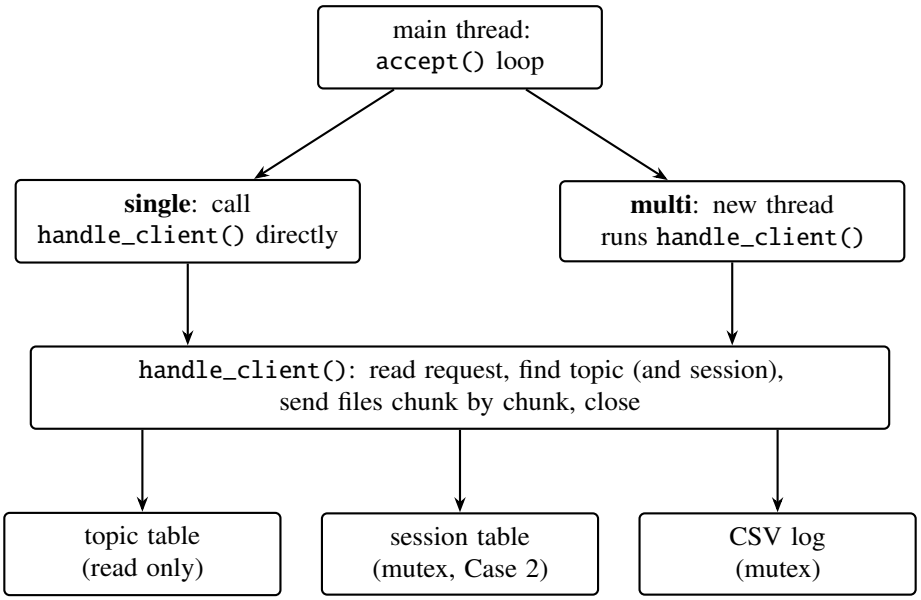


Figure 3: Structure of the server.

5.2 Shared data and locking

- The topic table is only read after start-up, so it needs no lock.
- The session table is protected by `session_lock`. A thread holds the lock only for short updates (find or add a session, update the high-water mark once per chunk), never during a socket or file operation.
- The CSV log has its own mutex.

5.3 Socket settings

Every connected socket has a send and receive timeout (`IO_TIMEOUT`), so a thread never waits forever for a dead peer. `SIGPIPE` is ignored, so writing to a closed connection returns an error instead of killing the server.

5.4 Client

In Part I, Case 1 and Case 2 the client is a single thread. In Case 2 Enhanced it runs one thread per stream (Section 8). In all parts the client:

- retries the first `connect()` up to `RETRY_LIMIT` times, `RETRY_DELAY` apart, so a client started just before the server is ready still works;
- creates <output_dir> and <output_dir>/<topic> (with any missing parent folders) only after the server has accepted the topic, so a request for an unknown topic leaves no empty folder behind. The folder is named with the topic exactly as it was given on the command line;
- rejects a received file name that contains `/` or is `.` or `..`, so files can only be written inside the topic folder;
- writes each file first as <name>`.part` and renames it to <name> when all its bytes have been received, so a file with its final name is always complete;
- exits with status 0 when the whole topic has been received, and with status 1 otherwise.

6 Fault Injection

Fault injection is done by the server, in every Part II case, in the thread that serves the connection.

When the decision is made. After the server has checked the request and sent the first reply line (**`OK`**, **`SESSION`** or **`RESUMED`**), and before it sends any file data, it computes R, the number of file bytes this connection is going to send:

- Case 1: the size of the whole topic;
- Case 2: the bytes from the checkpoint to the end of the topic;
- Enhanced: the remaining bytes of the files of this stream.

How the decision is made. The thread draws a random number u_1 in $[0,1)$. If $R > 0$ and $u_1 < p$, where p is the failure probability from the command line, this connection will be broken. In that case the thread draws a second number u_2 and sets the failure point $F = \lfloor u_2 R \rfloor$. So the failure happens after F bytes of file data, where F is equally likely to be anywhere in the transfer: before the first file, in the middle of a file, or just after a **`FILE`** line.

How the failure is done. The send loop counts the file bytes sent on this connection. If the next chunk would go past F, it is cut so that exactly F bytes have been sent. Then the server closes the socket abruptly: it sets `SO_LINGER` with a linger time of 0 and calls `close()`. This makes the kernel send a TCP reset

(RST) and throw away any data still waiting in the send buffer, which is close to what happens when a machine or link fails. The client sees a reset or end-of-file, and it may have received fewer bytes than the server sent. Because of this, the design never assumes that data passed to `send()` has reached the client.

```
/* after the first reply line, before any file data */
R      = file bytes this connection will send
failed = (R > 0) && (erand48(rng) < p)
F      = failed ? floor(erand48(rng) * R) : R
sent   = 0

/* inside the send loop, before sending n bytes (n <= CHUNK_SIZE) */
if (failed && sent + n > F) {
    n = F - sent;                 /* may be 0 */
    send n bytes;
    set SO_LINGER {1, 0}; close(socket);
    return;                       /* connection ends here */
}
```

Random numbers. Each connection gets its own random state for `erand48()`, made from the server seed and the connection number (the acceptor counts connections). Threads therefore share no random state and need no lock, and a run can be repeated with the same `seed=` value. For every connection the server logs whether a failure was planned, together with R and F.

Why one decision per connection. With this rule, p is simply the probability that a connection is broken, and a transfer needs $1/(1-p)$ connections on average in both Case 1 and Case 2. We first thought of deciding separately for every chunk. With that rule, Case 1 would have to get through thousands of chunks without a single failure, so for larger topics it would almost never finish. With $p = 1$ every connection fails, so Case 1 never completes and the client stops after `RETRY_LIMIT` attempts.

How the client notices a failure. Once connected, any of these counts as a failure: `recv()` returns 0 before **`END`** (end-of-file), `recv()` or `send()` returns an error such as `ECONNRESET`, or a receive times out. A failed `connect()` is simply retried. In Part I a failure after connecting ends the client with status 1. In Part II the client reconnects as described for each case.

7 Session Management (Case 2)

7.1 What each side stores

Client (in memory). The session id, the topic, and a checkpoint $C = (idx, offset)$: the file it is receiving and how many bytes of that file are already written to disk. The checkpoint is updated after every successful `write()` (at most 64 KiB at a time). When the last byte of a file is written, the `.part` file is renamed and the checkpoint moves to the start of the next file. The assignment describes a checkpoint as the topic, the current file name and the byte offset. Our checkpoint has the same content, but the file is identified by its index in the manifest, which names the file and also gives its place in the sending order. The client also remembers the name of the file it was writing, so it can check the name after a resume.

Server (session table). The session id, the topic, and a high-water mark $H = (sent_idx, sent_off)$: the furthest position the server has ever sent for this session. H is raised before each `send()` of file data, to the position just after that chunk; for an empty file it is raised to the next file before the **`FILE`** line is sent. Because H is raised before sending, the client can never hold data beyond H. The server also stores whether **`BYE`** has been received, and an owner number for each stream (Section 7.5).

So the client knows exactly what it has saved, and the server knows the most that could have been sent. The server uses H to check that a checkpoint reported by the client is possible. The client does not need to send any acknowledgements during the transfer.

7.2 Positions

A position (i, o) means "byte o of file i". Positions are compared first by i and then by o. Both sides always write a position in the same form: when all bytes of file i have been handled, the position becomes $(i+1, 0)$ (in Enhanced, $(i+k, 0)$) rather than $(i, size_i)$. A position with $i \geq nfiles$ means that everything has been sent. An empty file counts as handled when its **FILE** line is sent (server) or when the empty file has been created (client).

7.3 Message sequence

Figure 4 shows a transfer with one failure.

1. The client connects and sends **GET** <topic>.
2. The server creates a session with a new random 64-bit id (checked to be unique in the table), sets H to $(0, 0)$ (in Enhanced, H of stream s is set to $(s, 0)$, the first position of that stream), and replies **SESSION** <sid> <nfiles> <total>.
3. The client saves the id and sets $C = (0, 0)$. The server decides about a failure and starts sending files.
4. If the connection breaks, the server thread simply ends. The session stays in the table. The client closes its socket, keeps C, waits RETRY_DELAY and connects again.
5. The client sends **RESUME** <sid> <topic> <idx> <offset> with its checkpoint.
6. The server checks the request (Section 7.4). If it is valid, it replies **RESUMED** <sid> <idx> <offset>, makes a new failure decision for the remaining bytes, and sends file <idx> from byte <offset>, followed by the remaining files.
7. The client checks that the first **FILE** line has exactly its <idx> and <offset>, and, when the offset is not 0, the name of the file it was writing, opens <name>.part, truncates it to <offset> bytes, and appends the new data.
8. After the last file the server sends **END**. The client sends **BYE** and closes. The server marks the session as finished.

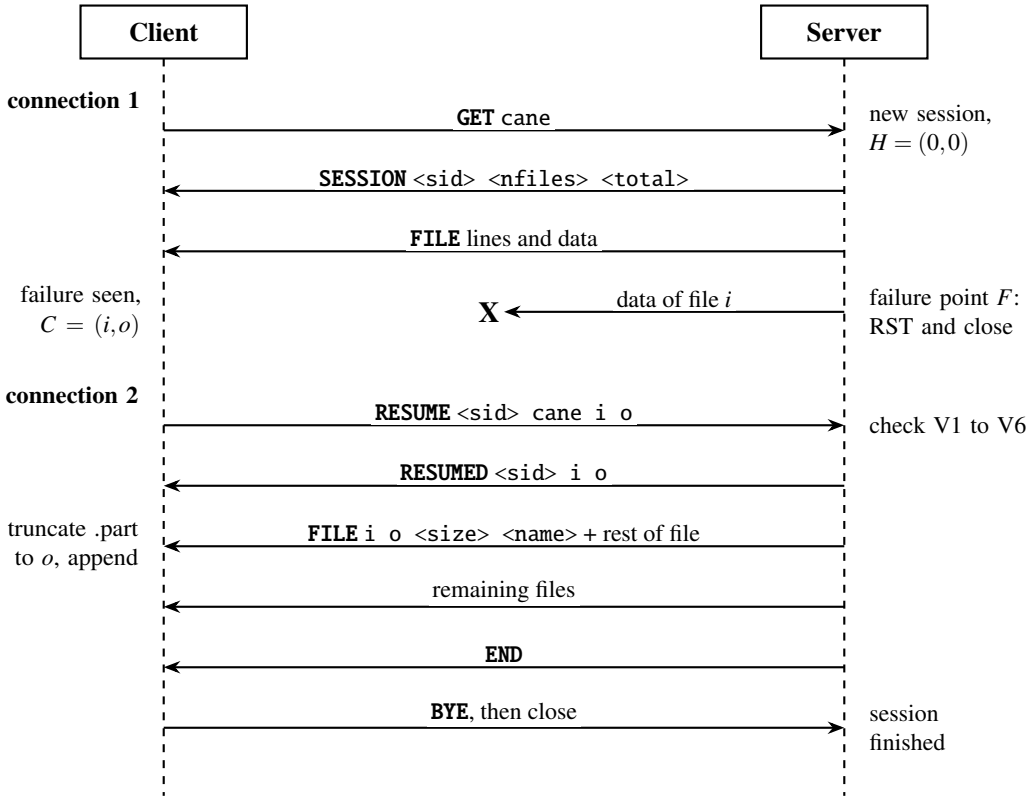


Figure 4: Case 2: a failure during file i and the resume from checkpoint (i, o).

7.4 Checking a RESUME request

The server checks a **RESUME** request in this order and stops at the first rule that fails. In Case 2 there is no stream field, so the rules are used with $stream = 0$ and $k = 1$.

Table 10: Checks on a RESUME request.

Rule	Check	If it fails
V1	The line has the right number of fields, the id is 16 hex digits, and the numbers are valid.	**ERR** 400
V2	The session exists, is not finished, and this stream has not already sent **BYE**.	**ERR** 410
V3	The topic is the same as the session's topic.	**ERR** 409
V4	(Enhanced) $stream < k$.	**ERR** 409
V5	Either $idx \geq nfiles$, or $idx \bmod k = stream$ and ($offset = 0$ or $offset < size_{idx}$).	**ERR** 409
V6	$(idx, offset) \leq H$ for that stream, i.e. the client does not claim data that was never sent.	**ERR** 409

If the checkpoint is at the end ($idx \geq nfiles$), the server replies **RESUMED** and then **END** straight away. This happens when the connection broke after the last byte but before **END** arrived.

A valid client never gets a 409 or 410. The client always reports what it really wrote, and that is never more than H. A session is only marked finished after the client has sent **BYE**.

7.5 A RESUME while the old connection is still active

With an injected fault the server thread closes its own socket, so it has already ended when the client reconnects. After a real network failure, however, a new **RESUME** can arrive while the old thread of the same stream is still running or is blocked in `send()`. The rule is that *the newest accepted connection owns the stream*:

1. Each stream has an owner number `gen[s]`. When the server accepts **GET** (stream 0) or a valid **RESUME**, it increases `gen[s]` under `session_lock`, and the new thread keeps the value it got.
2. The new connection is accepted at once. The server does not wait for the old thread to finish.
3. Before sending each chunk, and before sending **END**, a thread checks under the lock that `gen[s]` still equals its own value. If not, it has been replaced: it stops, closes its socket without sending anything more, and ends. The check and the raising of H are done in the same locked step, so a replaced thread never changes H or `done[s]`.
4. An old thread blocked in `send()` wakes up when that call fails (the client has closed the socket) or after `IO_TIMEOUT`, and then ends in the same way.
5. **BYE** is accepted only from the current owner.

This is safe because the check in V6 uses H at the moment of the **RESUME**, and H is always raised before data is sent. Whatever the old connection delivered, the client's checkpoint cannot be beyond H. After the new connection is accepted, only its thread sends data for that stream, and the client reads only from the new socket.

7.6 Client procedure

```
connect, send "GET <topic>", read SESSION
    (on failure: wait, then try again with GET)
C = (0, 0)
loop:
  receive frames until END or failure:
    FILE i o size name:
        check that (i, o) == C
        open name.part and truncate it to o bytes
        read data and write it; advance C after each write
        when the file is complete: rename to name, C = (i+1, 0)
    END:
        send "BYE", close, exit 0
  on failure:
    close the socket, attempts = attempts + 1
    if attempts > RETRY_LIMIT: exit 1
    wait RETRY_DELAY, connect again
    send "RESUME <sid> <topic> <C.idx> <C.offset>"
    expect RESUMED (on ERR: exit 1)
```

7.7 Why no saved data is sent again

- The client moves C forward only after a `write()` has succeeded, and the server always resumes exactly at C. Saved bytes are therefore never requested again, and no byte is skipped.
- Bytes that were sent but lost in the reset are after C, so they are sent again. These are the only redundant bytes in Case 2.
- The manifest does not change while the server runs, so an index and an offset refer to the same bytes on every connection.
- Truncating the `.part` file to o before appending removes any bytes that may have been written after the checkpoint.

8 Performance Enhancements (Case 2 Enhanced)

The enhanced version keeps everything from Case 2 (sessions, checkpoints, checks and fault injection) and adds parallel TCP connections.

8.1 Socket mode

We keep *blocking* sockets with the 30 second timeouts. Each connection has its own thread, so while one thread is blocked on its socket, the other threads continue with theirs. Non-blocking sockets would need an event loop with `poll()` that handles many sockets in one thread. That mainly helps with thousands of connections, while we have tens, and it would not fit the one-thread-per-client design used in the rest of Part II.

8.2 Parallel TCP connections

- **Number of connections.** The client asks for k connections (`conns=`, default 4). The server allows at most `MAX_STREAMS` = 8 and returns the value it accepts in **`SESSION`**. We will try k = 1, 2, 4 and 8.
- **How the work is split.** File i is sent on stream i mod k, and each stream sends its files in increasing order. A topic is a set of many separate image files, and consecutive files go to different streams, so each stream gets a similar share of the files. The streams do not have to coordinate with each other. How evenly the bytes are spread for the chosen topic will be checked from the logs.
- **How the data is put together.** Every file travels whole on one stream and is written to its own file, so no bytes have to be reassembled. Two streams never write to the same file. The topic is complete when all k streams have received **`END`**.
- **How a failed connection is recovered.** Each stream has its own checkpoint on the client and its own high-water mark on the server. If one stream fails, only that stream reconnects with **`RESUME`**. The other streams are not affected.

On the server, each stream connection is an ordinary connection handled by its own thread, exactly as in Case 2. On the client, the main thread opens stream 0 with **`GET`** `<topic> <k>` and reads **`SESSION`**. It then starts one thread per stream. Stream 0 continues on the connection that is already open. Streams 1 to $k-1$ connect and send **`RESUME`** with the checkpoint $(stream, 0)$. This first **`RESUME`** passes check V6 because the server set H of stream s to $(s, 0)$ when it created the session, so a stream that has received nothing yet can always attach. Each thread runs the Case 2 receive and retry loop for its own stream. If stream 0 fails before **`SESSION`** arrives, no session exists yet, so the client simply sends **`GET`** again.

When the transfer is complete. A stream is finished when the client has received its **`END`**, renamed all its files and sent **`BYE`**. Every stream goes through this, including a stream with no files ($s \geq nfiles$), which gets **`RESUMED`** followed directly by **`END`**. The transfer is complete only when *all* k streams are finished. The client's main thread waits for all stream threads with `pthread_join()` and exits with status 0 only if every stream finished. If any stream uses up its retries or receives **`ERR`**, it sets a shared flag, the other threads stop, and the client exits with status 1, even if some streams had already finished. On the server, the session is marked finished only when **`BYE`** has been received for every stream (`done[s]` set for all s). Until then, a finished stream is rejected by V2, but an unfinished stream can still resume.

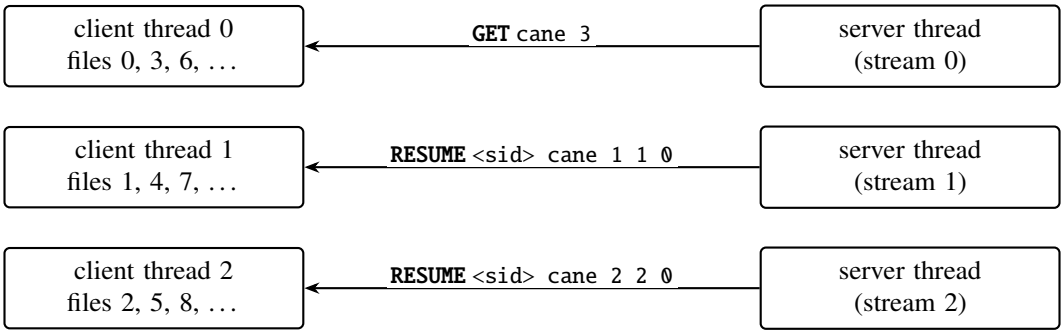


Figure 5: Case 2 Enhanced with $k = 3$. Each stream has its own TCP connection and its own thread on both sides. The arrows show the direction of the file data.

8.3 How the improvement will be shown

Case 2 is the baseline. We will run the enhanced version with k = 1, 2, 4 and 8 under the same conditions ($N = 8$ clients, p = 0 and 0.3). With $k = 1$ the enhanced version behaves exactly like Case 2, so any change in throughput or completion time as k grows comes from the parallel connections. Note that with k streams there are k connections that can each fail with probability p. We will take this into account in the analysis.

9 Performance Measurement

9.1 What is measured

Server, for every connection. The start time (right after the request line has been read) and the end time (right after **END** has been sent, or when the connection breaks or fails), both from `clock_gettime(CLOCK_MONOTONIC)`. It also records the number of bytes passed to `send()`, counted separately for protocol lines and file data, and whether a failure was injected. This is one line in the server CSV file.

Client, for every transfer. The time from the first `connect()` until all files are saved, including reconnections, the number of connections used, the bytes received, and whether the transfer succeeded. This is one line in the client CSV file.

All times are measured on one machine at a time, so the clocks of different machines do not need to agree.

9.2 Metrics

For a run with N clients that each download a topic of size S bytes:

Table 11: Metrics.

Metric	Definition
Run time W	Latest end time minus earliest start time over all connections of the run (server clock).
File bytes sent B_{file}	Sum of all file data bytes sent by the server, including data sent again after failures.
Total bytes sent B_{tx}	B_{file} plus the bytes of all protocol lines.
Useful bytes B_{use}	$N \times S$, the data that ends up in the clients' files.
Redundant bytes B_{red}	$B_{file} - B_{use}$, i.e. file data sent more than once.
Overhead	$(B_{tx} - B_{use})/B_{use}$.
Aggregate file transfer rate	B_{file}/W in MB/s (1 MB = 10^6 bytes). This is the main Part I metric. The rate including protocol lines, B_{tx}/W, is also reported.
Goodput	B_{use}/W in MB/s.
Completion time	Per-client transfer time; we report the average and the maximum.

9.3 Experiments

Table 12: Planned experiments. Clients are split equally between nodes B and C.

Experiment	What changes	Result
Part I	N = 2, 4, 8, 16, 32; single and multi-threaded	Aggregate file transfer rate against N (table and graph)
Case 1 and Case 2	p = 0, 0.1, 0.2, 0.3, 0.5, 0.7 with $N = 8$	Total, useful and redundant bytes, completion time, goodput; goodput against overhead
Compared with Part I	$p = 0$, $N = 8$	Case 1 and Case 2 against Part I multi-threaded
Enhanced	k = 1, 2, 4, 8 with $N = 8$ and p = 0, 0.3	Goodput and completion time compared with Case 2

All clients in a run request the same topic, whose number of files and size will be given in the report. Every setting is run 5 times and we report the mean and the standard deviation. After each run the downloaded files are compared with the originals (`diff -r`) to make sure the transfer was correct.

9.4 Output files

The raw CSV files and the graphs will be stored in `Results/`, using the naming pattern suggested in the assignment:

- Part I: `Part1_SingleThreaded_throughput.csv`, `Part1_MultiThreaded_throughput.csv`, `Part1_Comparison_throughput.png`;
- Case 1: `Part2_Case1_throughput.png`, `Part2_Case1_overhead.png`;
- Case 2: `Part2_Case2_throughput.png`, `Part2_Case2_overhead.png`;
- Enhanced: `Part2_Case2Enhanced_throughput.png`, `Part2_Case2Enhanced_overhead.png`.

9.5 What we expect

These are expectations to check, not results. The single-threaded server should give about the same aggregate file transfer rate for any N, because it serves one client at a time. The multi-threaded server should improve as N grows, until the network or the machines become the limit. In Case 1 each failure wastes on average half of the topic, while in Case 2 it wastes only the data lost in the reset. The gap between the two should therefore grow quickly as p increases.

10 Additional Features

We keep the extra features small and only add those that the testing or the experiments need.

Table 13: Additional features.

Feature	Reason
Optional `key=value` arguments	Lets us change the server mode, seed and number of connections for experiments without breaking the required command line.
CSV measurement output	Gives the raw data for all tables and graphs.
Seed for fault injection	A run with failures can be repeated exactly for debugging.
`.part` files renamed on completion	A file with its final name is always complete, even after failures.
Checks on topic and file names	The server only serves topics from its table, and the client never writes outside the topic folder.

11 Code Structure

11.1 Folders and files

Each part is in its own folder with its own sources and `Makefile`, as required. Files with the same name have the same role in every folder. Each part starts as a copy of the previous one and adds to it.

```
FinalSubmission_Group_016_024/
    FinalReport_Group_016_024.pdf
    README.md
    Part1/                  Makefile  server.c  client.c
                            topics.c  topics.h  common.c  common.h
    Part2_Case1/            same files, plus fault.c  fault.h
    Part2_Case2/            same files, plus session.c  session.h
    Part2_Case2_Enhanced/   same files as Part2_Case2 (streams added)
    Results/                CSV files, graphs, run and plot scripts
```

11.2 Main functions

Table 14: Source files and their main functions.

File	Contents	Main functions
`common.c`	Socket helpers, line reader, timing, CSV writing	`send_all()`, `read_line()`, `read_some()`, `set_timeouts()`, `now()`, `csv_write()`
`topics.c`	Topic table	`load_topics()`, `find_topic()`
`server.c`	Arguments, listening socket, single or multi-threaded loop, request handling, send loop	`main()`, `serve_single()`, `serve_multi()`, `handle_client()`, `send_files()`
`client.c`	Arguments, folders, request, receive loop, retry loop, stream threads (Enhanced)	`main()`, `connect_server()`, `receive_files()`, `stream_thread()`
`fault.c`	Fault decision (Part II)	`fault_init()`, `fault_limit()`
`session.c`	Session table (Case 2 and Enhanced)	`session_new()`, `session_check()`, `session_raise_hwm()`, `session_finish()`

11.3 How the parts fit together

The key function is `send_files(conn, topic, start, stream, k)`, which sends the files of a topic from a start position. It is the same loop in every part. The later parts only add calls inside it:

- **Part I**: start position $(0,0)$, $k = 1$, no extra calls. The single-threaded and multi-threaded servers both use it through `handle_client()`.
- **Case 1**: `fault_init()` before the loop and `fault_limit()` before each chunk.
- **Case 2**: the start position comes from **`RESUME`**, and `session_raise_hwm()` is called before each chunk.
- **Enhanced**: the loop steps through the files with step k, starting at the stream's position.

On the client, `receive_files()` is also shared. In Case 1 a retry loop is added around it, and in Case 2 the checkpoint is updated inside it. In Enhanced, each stream thread runs the same loop for its own stream.

11.4 Build

Each `Makefile` builds `server` and `client` with `gcc -std=gnu11 -O2 -Wall -Wextra -pthread` and has a `clean` target. Running `make` in a folder produces exactly these two executables, and the code must compile without any warnings, as the submission rules require. `README.md` gives the build and run commands for every case and describes the fault-injection mechanism.

12 Design Thought Process

12.1 Part I

- **Topic lookup.** We considered a hash table, a sorted array, and scanning the folder on every request. With only ten topics, a sorted array is simple and fast enough. It also gives a fixed file order, which Case 2 needs. Scanning on every request could return files in a different order each time.
- **Message format.** We considered a binary header, sending the topic as one archive, and text lines. We chose text lines followed by raw data with a length. The required request **`GET`** `<topic>` is already text, lines are easy to debug, and the length tells the client where each file ends.
- **Multi-threaded design.** We considered a thread pool, separate threads for reading files and for sending, and one thread per client. We chose one thread per client. It is simple, each transfer stays a straight loop, and it is what Part II requires. With tens of clients, creating a thread per client costs very little compared with sending a topic.

12.2 Case 1

- **Where to inject faults.** On the server, as the assignment asks. It also means that all fault decisions are logged in one place.
- **How often to decide.** Per chunk, per file, or per connection. We chose per connection, for the reason given in Section 6: otherwise Case 1 would almost never finish a large topic.
- **How to break the connection.** A normal `shutdown()` or an abrupt close with RST. The RST is closer to a real failure because data in flight is lost.

12.3 Case 2

- **How often to checkpoint.** After every file, after every few KB, or after every write. Updating an in-memory checkpoint after every write costs nothing and gives the least resending.

- **Who decides the resume point.** If the server resumed from what it had sent, data lost in the reset would be skipped. If the server stored nothing, it could not check the client's claim. We let the client report what it saved and let the server check it against the high-water mark.
- **Session identification.** The client's IP and port cannot be used, because the port changes on every reconnect and several clients run on the same machine. So the server gives each session a random id.

12.4 Case 2 Enhanced

- **Splitting the work.** We considered splitting single files into byte ranges, a shared queue of files, and a fixed split by file index. Byte ranges only pay off when a topic has few, large files. Here each topic is a set of many separate images, so there is enough parallel work at the file level. A shared queue balances the load a little better but makes recovery harder. The fixed split is simple, and each stream can resume on its own just like Case 2.
- **Socket mode.** Blocking sockets with one thread per connection, as explained in Section 8.

13 Assumptions and Limitations

- The dataset files do not change while the server is running. If a file cannot be opened, the server sends `ERR` 500. If a file turns out to be shorter than expected while sending, the connection is closed.
- Sessions are kept only in the server's memory. The assignment's faults are broken connections, so this is enough. If the server process itself is restarted, old sessions are lost and the client exits with an error.
- Topic names do not contain spaces, and file names do not contain newlines. This holds for the given dataset, whose ten folder names are plain lower-case words. Entries that break this rule are skipped with a warning.
- Only IPv4 is supported.